

Tile-Engine $\leftrightarrow$ Dispatcher Parity: Complete Design

The Core Problem

ComposableKernel (CK) has **two systems** that describe the same hardware kernels — GEMM operations on AMD GPUs — but in completely different vocabularies:

System	Role	Format
Tile Engine (TE)	Old, per-binary-per-kernel generator	JSON config + benchmark_gemm_universal_* executables
Dispatcher	New, one-.so-many-kernels runtime	C++ KernelKey struct + Python binding

This project builds the **bridge**: a translator + proof system that demonstrates the dispatcher reproduces the Tile Engine — same kernel shapes, same math, same performance — so the dispatcher can replace the TE in production.

The Vocabulary Problem (The Central Trap)

The two systems describe the same hardware parameter using **different names**. The most dangerous trap:

Tile Engine says:	Dispatcher says:	
"warp_m/n/k" =	"wave_shape.m/n/k"	← count of warps/waves per block
"warp_tile_m/n/k" =	"warp_tile_m/n/k"	← MFMA shape per warp (same name!)
Scheduler "default" =	"auto"	← string form differs

Why this is catastrophic if you get it wrong: Swapping warp counts and warp_tile produces a config that is *valid-looking* (passes validation) but launches the *wrong shape kernel*. Answers are wrong, no error message, hours of debugging. The code documents this in `_Tile`'s docstring at `te_to_dispatcher.py:131`:

```
@dataclass(frozen=True)
class _Tile:
    # NOTE: Naming trap – TE uses "warp_m/n/k" to mean wave counts per block
    # (how many waves/warps tile the block). The dispatcher calls these same
    # values "wave_shape.m/n/k". What the dispatcher calls "warp_tile" is the
    # per-warp MFMA shape (tile_m/n/k per wave). They map one-to-one but the
    # vocabularies are swapped; mixing them produces valid-looking but wrong kernels.
    tile_m: int; tile_n: int; tile_k: int
    warp_m: int; warp_n: int; warp_k: int          # = wave COUNT per block
    warp_tile_m: int; warp_tile_n: int; warp_tile_k: int  # = MFMA shape per wave
```

System Architecture (Data Flow)

[illegible]

- ▼

{

There are **two different name strings** and mixing them is a silent failure:

1. Registry Identifier (encode_identifier)

ΛΛΛΛ

Used to find the .hpp header on disk and find the TE benchmark binary. Built from **raw TE strings**:

AAAAAAAA

The `_preshuffle` suffix adds a third wrinkle: `unified_gemm_codegen.py` appends `_preshuffle` to the file name for `preshufflev2` pipeline. Both `te.kernel_name()` and `sweep_runner.kernel_name()` must replicate this or file-based

lookups fail.

Translation Layer Deep Dive ([te_to_dispatcher.py](#))

The Mapping Tables

```
# Scheduler canonicalization (the most important mapping)
_SCHEDULER_CANON = {
    "intrawave": "intrawave",
    "interwave": "interwave",
    "default": "auto", # ← TE "default" → dispatcher "auto"
    "auto": "auto",
}

# Output dtype promotion (fp8 is too narrow for C matrix)
_OUTPUT_DTYPE = {"fp8": "fp16", "bf8": "fp16"}

# Accumulator type per input dtype
_ACC_DTYPE = {"fp16": "fp32", "bf16": "fp32", "fp8": "fp32", "int8": "int32"}

# Double-buffering is pipeline-driven, not a user flag
_DOUBLE_BUFFER_PIPELINES = {"compv4", "preshufflev2"}

# These pipelines exist in TE but have no dispatcher codegen path
_UNSUPPORTED_PIPELINES = frozenset({"compv1", "compv2", "preshufflev1"})

# Unsupported trait triples (compute pipeline + interwave = illegal)
_UNSUPPORTED_TRAITS = frozenset(
    (p, e, "interwave")
    for p in ("compv3", "compv4", "compv5", ...)
    for e in ("cshuffle", "default")
)
```

The Dual-Store in `_build_config`

The translator stores the same data **twice** — once in canonical dispatcher form, once in raw TE form — to serve two different downstream consumers without any re-mapping:

```
return {
    "_te": {
        "pipeline": pipeline, # ← for codegen (drive_codegen.py uses these)
        "scheduler": scheduler, # raw: "default"
    },
    "algorithm": {
        "pipeline": _PIPELINE_CANON[pipeline], # canonical: "compv3"
        "scheduler": _SCHEDULER_CANON[scheduler], # canonical: "auto"
        "double_buffer": pipeline in _DOUBLE_BUFFER_PIPELINES, # derived flag
        "preshuffle": pipeline in ("preshufflev1", "preshufflev2"),
    },
}
```

The `split_k` Guard

```
# split_k is cast to uint8_t in C++ oracle (line 69 of cpp_identifier_oracle.cpp).
# Values > 255 wrap silently → Python/C++ identifier mismatch.
if not (1 <= split_k <= 255):
    raise TranslationError(f"split_k={split_k} out of range [1, 255]")
```

This is an example of the "always explicit, never rely on defaults" philosophy baked into the design.

Identifier Parity (`identifier.py` + `cpp_identifier_oracle.cpp`)

The identifier must be **byte-for-byte identical** between Python and C++. The design makes this provable by ensuring the Python function is **pure concatenation** with no mapping logic:

```
def encode_identifier(cfg):
    # All strings are ALREADY canonical (translator did the mapping once)
    # So this is just assembly – no conditionals on string values
    parts = [
        f"{sig['dtype_a']}_",
        f"{sig['layout_a']}{sig['layout_b']}{sig['layout_c']}_",
        f"{alg['pipeline']}_",
        f"{alg['epilogue']}_",
        f"{alg['scheduler']}_",
        f"_{cpp_bool(alg['pad_m'])}_",      # "True"/"False" matching C++
        ...
        f"{alg['tile_m']}{alg['tile_n']}{alg['tile_k']}"
        f"_{alg['warp_m']}{alg['warp_n']}{alg['warp_k']}"
        f"_{alg['warp_tile_m']}{alg['warp_tile_n']}{alg['warp_tile_k']}"
    ]
    # Optional suffixes in EXACT C++ order:
    if split_k > 1:      identifier += f"_splitk{split_k}"
    if op not PassThru:  identifier += f"_{op}"
    if num_d > 0:        identifier += f"_d{num_d}"
    if sparse:           identifier += "_sparse"
    if preshuffle:       identifier += "_preshuffle"
```

`check_identifier_parity.py` runs the C++ oracle (`g++ cpp_identifier_oracle.cpp`) and asserts the outputs match for every config in a file. This is **Stage 1** — always runs on CPU, no GPU needed.

Codegen Bridge (`drive_codegen.py`)

The TE codegen (`unified_gemm_codegen.py`) expects a specific JSON shape with **flat lists** and **raw TE strings**. `drive_codegen.py` builds a "minimal config" — one value per parameter — from the already-translated dispatcher config:

```
def _minimal_te_config(cfg):
    te = cfg["_te"]      # raw TE strings
    alg = cfg["algorithm"]
    return {
        "block_size": alg["block_size"],    # must be explicit or codegen defaults to 256
        "tile_config": {
            "tile_m": [alg["tile_m"]],      # flat list, not {"values": [...]}
            ...
        },
        "trait_config": {
            "pipeline": [te["pipeline"]],    # raw "compv3", NOT canonical
            "scheduler": [te["scheduler"]],  # raw "default", NOT "auto"
            ...
        }
    }
```

Output: `generated/parity_single/gemm_<te_kernel_name>.hpp` — a standalone header that exposes the compiled kernel.

Metric	Value	Rationale
Perf measurement	median of 10 runs	Single runs are meaningless on GPUs with boost state
Warmup	3 invocations	Matches <code>stream_config.cold_niters_=3</code> in dispatcher

Phase 2: Sweep Infrastructure

Sweep Runner (sweep_runner.py)

Generalizes Phase 1's single-config parity check to the full Cartesian product:

```
for each config_file:
  for each translated config:
    for each (M, N, K) problem size:
      1. drive_codegen
      2. build_harness
      3. run_harness -verify=1
      4. write_row_to_Parquet_immediately (crash-safe)
      5. on_resume: skip (identifier, M, N, K) already in Parquet
```

Schema per row: {config_file, config_index, identifier, kernel_name, datatype, M, N, K, verdict, tflops, error_msg, stage_failed, ts}

Compare Report (compare_report.py)

Joins dispatcher and TE Parquet files on (identifier, M, N, K), computes $\text{delta\%} = (\text{disp} - \text{te}) / \text{te} * 100$, produces Markdown or HTML with per-shape rows and rolled-up summary tables.

Validation Rules (What Gets Rejected)

The translator enforces these at translation time — fast failure before codegen:

Rule	Reason
<code>tile_m % (warp_m * warp_tile_m) != 0</code>	Block tile must divide evenly into waves
<code>pipeline in {"compv1","compv2","preshufflev1"}</code>	No codegen path in <code>unified_gemm_codegen.py</code>
<code>(compv3/4/5, *, "interwave")</code>	Hardware constraint: compute pipelines require intrawave
<code>split_k > 255</code>	<code>uint8_t</code> overflow in C++ oracle

The Five Design Principles

1. Map exactly once, store twice

The translator (`te_to_dispatcher.py`) is the only place TE strings are converted to dispatcher canonical form. After translation, the config carries both forms: `- cfg["_te"]` — raw strings, consumed by codegen `- cfg["algorithm"]` — canonical strings, consumed by `encode_identifier`

No downstream code re-maps strings.

2. Always emit every field explicitly

Padding fields (`pad_m`, `pad_n`, `pad_k`) are always set, never defaulted. The two stacks have *different* defaults for padding, so omitting any field causes the downstream code to see different values. Discovered via padding-path bugs on non-tile-aligned problem sizes.

3. The identifier is the contract

Stage 1 (identifier parity) runs on every invocation, even without a GPU. If codegen names the header "kernel_A" but the runtime looks for "kernel_B", the dispatcher returns null silently. The identifier test catches this before any GPU code runs.

4. Numerical stages are per-stack self-consistency, not cross-stack

Each stack verifies against its own CPU fp32 reference. If both pass, neither has a kernel computation bug. True cross-stack comparison (feed dispatcher C as TE reference) would require a shared buffer protocol — out of scope. Performance (TFLOP/s) IS cross-stack comparable because it is input-independent.

5. Codegen is a black box you drive, not modify

`drive_codegen.py` constructs a minimal TE config that exactly matches what codegen expects (flat lists, raw TE strings, all fields explicit) rather than modifying codegen to accept dispatcher format. This keeps the port surface area minimal and avoids breaking the existing TE codegen path.

GPU-Verified Configurations (gfx942 / MI300X)

Config	Status	Notes
<code>single_fp16_rcr.json</code> (compv3/intrawave, no padding)	GPU-verified (Stages 1–3)	512³/1024³/2048³: 17.9 / 84.7 / 269 TFLOP/s
<code>padding_fp16_rcr.json</code> (<code>pad_m/n/k=true</code>)	GPU-verified (Stages 1–3)	257×257×56, 513×511×40 pass
<code>single_bf16_rcr.json</code>	GPU-verified (Stages 1–2)	512³: 17.9 TFLOP/s; 1024³: 85.9 TFLOP/s
<code>single_fp8_rcr.json</code>	GPU-verified (timing-only)	Numerical verify needs <code>CK_TILE_USE_CUSTOM_DATA_TYPE</code>
<code>single_int8_rcr.json</code>	GPU-verified (Stages 1–2)	int32 accumulator; 512³: 25.9 TFLOP/s
<code>single_fp16_rcr_splitk.json</code> (<code>split_k=4</code>)	GPU-verified (Stages 1–2)	<code>split_k</code> appended to identifier; runtime param

Current Status

Phase	Tasks	Status
Phase 1	T1.1–T1.7	Complete; GPU-verified on gfx942
Phase 2	T2.1, T2.3, T2.6	Implemented; 220 tests passing on CPU
Phase 2	T2.2 (multi-kernel .so)	Code written; .so not yet compiled
Phase 2	T2.4–T2.5 (full fp16/bf16 sweep)	Blocked on T2.2 .so build on gfx942